

DTRU

Algorithm Specifications and Supporting Documentation

(version 1.0)

Yunlei Zhao, Hengchuan Zou, Songlin Li, Jieyu Zheng,
Xiaowen Hu, Hanyu Wei, Weizhi Ao, Jian Weng, Zhaohui Cheng,
Yanbin Pan and Wenbo Guo

June 29, 2026

DTRU: A Versatile, Compact, Simple, and Robust NTRU KEM with Double E_8 Encoding

1 Algorithm Description

DTRU is a versatile, compact, simple, and robust NTRU-based key encapsulation mechanism (KEM) that achieves provable IND-CCA security under the NTRU and Ring-LWE assumptions. Our primary instantiation employs tricyclotomic rings $(\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1))$, enabling seamless support for multiple classical security strength levels. Additionally, we provide DTRU-Prime instantiation on LPPNF (large-Galois-group prime-degree prime-ideal number field $(\mathbb{Z}_q[x]/(x^n - x - 1))$) as a conservative defense against algebraic attacks and DTRU-Light instantiation on power-of-two cyclotomic rings $(\mathbb{Z}_q[x]/(x^n + 1))$ to provide a more lightweight option. Our parameter selection adopts a conservative approach, refraining from sparse noise distributions and utilizing centered binomial distributions with parameter at least 2 for most parameters.

1.1 Double E_8 Encoding

Our construction aims to encode $n/4$ bits into n coefficients. This encoding requirement necessitates a binary code that maps 1 bit to 2 bits on average. When combined with NEV’s [ZFY23] repetition code, this yields a 1-to-4 mapping.

Building upon this foundation, we further enhance error correction by increasing the encoding dimension. Specifically, we employ a four-dimensional sublattice of the E_8 lattice to construct an extended Hamming-style code that maps 4 bits to 8 bits, achieving a minimum Hamming distance of 4—an optimal property among binary [8,4] codes. The combination of this extended Hamming-style code with NEV’s repetition encoding yields a double E_8 encoding that effectively maps 4 bits to 16 coordinates while maintaining modest decoding complexity.

1.1.1 Message Encoding

The core component of our double E_8 encoding is the E_8 code, which is constructed from a four-dimensional sublattice of the E_8 lattice. To encode a message $m \in \{0, 1\}^4$, we compute the codeword $u = mH \bmod 2$, where H is the

generator matrix defined as follows:

$$H = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

This E_8 code achieves a minimum Hamming distance of $d = 4$, which is optimal among binary $[8,4]$ codes.

The double E_8 encoding is constructed by applying NEV's repetition encoding to the E_8 codewords. Specifically, a message $m \in \{0,1\}^4$ is first encoded as $u = mH \bmod 2$, then repeated to form $(u, u) \in \{0,1\}^{16}$, effectively mapping 4 bits to 16 coordinates. This combination yields a code of length 16 and dimension 4 with minimum Hamming distance $d = 8$.

In our encryption algorithm, the double E_8 code is scaled by $\frac{q+1}{2}$ to operate over $\mathbb{Z}_q$. We can establish that the minimum Euclidean distance between any two distinct scaled codewords is at least $\sqrt{2}(q-1)$. This distance property arises because for any two distinct codewords $\frac{q+1}{2}(u_1, u_1)$ and $\frac{q+1}{2}(u_2, u_2)$, the difference $(u_1, u_1) - (u_2, u_2)$ in $\mathbb{Z}_2^{16}$ is a nonzero codeword of the double E_8 code with Hamming weight at least 8. Considering the Euclidean norm in $\mathbb{Z}_q$, we obtain

$$\left\| \frac{q+1}{2}((u_1, u_1) - (u_2, u_2)) \right\|_{q,2} \geq \frac{q-1}{2}\sqrt{8} = \sqrt{2}(q-1).$$

The complete encoding procedure for a message $m \in \{0,1\}^\ell$ with $\ell \leq n/4$ is formalized in Algorithm 1. The algorithm applies the double E_8 scheme to produce a vector in $\{0,1\}^n$, which is then mapped to a polynomial in R_q . When $\ell < n/4$, the remaining bits are padded with zeros. The final multiplication by $(1 + x^{n/2})$ implements the repetition encoding, replicating coefficients from the first half to the second half to realize the double E_8 structure.

1.1.2 Message Decoding

Our decoding algorithm, formalized in Algorithm 2, is designed based on the maximum likelihood decoding principle. Given a received polynomial $t \in R_q$, the algorithm processes it in pieces of 16 coefficients. For each piece, the objective is to find a codeword $\frac{q+1}{2}(u, u)$ that minimizes the squared Euclidean distance $\|v - \frac{q+1}{2}(u, u)\|_{q,2}^2$, where v is the 16-dimensional vector extracted from t .

To identify the most likely transmitted message, we evaluate the distance metric for all possible codewords. For each candidate codeword, the total distance is obtained by summing the relevant entries from the distance matrix according to the codeword's structure. The message $m \in \{0,1\}^4$ corresponding to the codeword that minimizes this total distance is selected as the decoded message.

Algorithm 1 Encode

Require: the message $m \in \{0, 1\}^\ell$, $\ell \leq n/4$ **Ensure:** the encoded polynomial $w \in R_q$

```
1: for  $i = 0$  to  $n/16 - 1$  do
2:    $(w_{8i}, w_{8i+1}, \dots, w_{8i+7}) \leftarrow (m_{4i}, m_{4i+1}, \dots, m_{4i+3}) \cdot H \bmod 2$ 
3: end for
4:  $w \leftarrow (w_0 + w_1x + \dots + w_{n/2-1}x^{n/2-1}) \cdot (1 + x^{n/2})$  in  $R_q$ 
5: return  $w$ 
```

Algorithm 2 Decode

Require: $t = t_0 + t_1x + \dots + t_{n-1}x^{n-1} \in R_q$ **Ensure:** $m \in \{0, 1\}^\ell$

```
1: for  $i = 0$  to  $n/16 - 1$  do
2:    $v \leftarrow (t_{8i}, t_{8i+1}, \dots, t_{8i+7}, t_{8i+1+n/2}, \dots, t_{8i+7+n/2}) \in \mathbb{Z}_q^{16}$ 
3:   for  $j = 0$  to  $7$  do
4:      $\text{dis\_matrix}[j][0] \leftarrow \|v_j\|_{q,2}^2 + \|v_{j+8}\|_{q,2}^2$ 
5:      $\text{dis\_matrix}[j][1] \leftarrow \|v_j - \frac{q+1}{2}\|_{q,2}^2 + \|v_{j+8} - \frac{q+1}{2}\|_{q,2}^2$ 
6:   end for
7:   for  $j = 0$  to  $15$  do
8:      $(u_0, \dots, u_7) \leftarrow (j_0, j_1, j_2, j_3) \cdot H$   $\triangleright j = (j_0j_1j_2j_3)_2$ 
9:      $\text{cost}[j] \leftarrow \sum_{k=0}^7 \text{dis\_matrix}[k][u_k]$ 
10:  end for
11:   $k \leftarrow \arg \min \text{cost}$ 
12:   $(m_{4i}, m_{4i+1}, m_{4i+2}, m_{4i+3}) \leftarrow (k_0, k_1, k_2, k_3)$   $\triangleright k = (k_0k_1k_2k_3)_2$ 
13: end for
14: return  $m$   $\triangleright$  Trim  $m$  if  $\ell < n/4$ 
```

The correctness of this decoding procedure is guaranteed by the minimum distance property of the scaled double E_8 code. With a minimum Euclidean distance of $\sqrt{2}(q-1)$, if the noise magnitude satisfies

$$\left\| v - \frac{q+1}{2}(u, u) \right\|_{q,2} < \frac{q-1}{\sqrt{2}},$$

the closest codeword to the received vector must be the transmitted codeword, ensuring correct decoding.

1.2 Algorithms of DTRU.PKE with Double E_8 encoding

The algorithms for key generation, encryption, and decryption in our DTRU public key encryption scheme are formally presented in Fig. 1.

The DTRU.PKE scheme follows a standard NTRU-based encryption framework with several key innovations. The algorithm flow consists of three main components:

Key Generation: The scheme generates a key pair $(\text{pk}, \text{sk}) = (h, f)$, where $f = pf' + 1$ is constructed to ensure invertibility in the ring R_q . The parameter p

Figure 1: Algorithms of DTRU.PKE

DTRU.PKE.KeyGen:	DTRU.PKE.Enc(pk = h, m ∈ {0, 1} ^ℓ):
1 : $f', g \leftarrow (\Psi_f, \Psi_g)$	1 : $r, e \leftarrow (\Psi_r, \Psi_e)$
2 : $f \leftarrow pf' + 1$	2 : $w \leftarrow \text{Encode}(m)$
3 : if f is not invertible in R_q	3 : $c \leftarrow \lfloor \frac{q_2}{q}(hr + e + \frac{q+1}{2}w) \rfloor \bmod q_2$
4 : go to step 1 and resample f'	4 : return c
5 : endif	
6 : $h \leftarrow gf^{-1}$ in R_q	DTRU.PKE.Dec(sk = f, c):
7 : return (pk, sk) = (h, f)	1 : $c' \leftarrow \lfloor \frac{q}{q_2} \cdot c \rfloor \bmod q$
	2 : $m' \leftarrow \text{Decode}(c'f)$
	3 : return m'

is chosen based on the ring structure: $p = 2$ for tricyclotomic rings and LPPNF, or $p = 1 - x^{n/2}$ for power-of-two cyclotomic rings. This parameter selection is specifically designed to minimize the error rate, as analyzed in the subsequent error rate analysis.

Encryption: The encryption process samples noise polynomials r and e , encodes the message m using double E_8 encoding, and computes the ciphertext. A key feature is the compression operation $\lfloor \frac{q_2}{q}(\cdot) \rfloor \bmod q_2$, which reduces ciphertext size by compressing from modulus q to q_2 .

Decryption: Decryption reverses the compression to recover the original modulus, then uses the secret key f to recover the encoded message, which is decoded back to the plaintext.

1.3 Algorithms of DTRU.KEM

Figure 2: Algorithms of DTRU.KEM

DTRU.KEM.KeyGen:	DTRU.KEM.Decaps:
1 : (pk, sk') ← DTRU.PKE.KeyGen(1 ^κ)	1 : m' ← DTRU.PKE.Dec(sk', c)
2 : s ← {0, 1} ^κ	2 : (K', ρ') ← H(ID(pk), m')
3 : return (pk, sk = (sk', pk, s))	3 : $\tilde{K} \leftarrow H_1(\text{ID}(\text{pk}), s, c)$
DTRU.KEM.Encaps:	4 : if DTRU.PKE.Enc(pk, m'; ρ') ≠ c
1 : m ← M	5 : return $\tilde{K}$
2 : (K, ρ) ← H(ID(pk), m)	6 : else
3 : c ← DTRU.PKE.Enc(pk, m; ρ)	7 : return K'
4 : return (c, K)	8 : endif

To construct the DTRU.KEM scheme, we employ the $\text{FO}_{ID(\text{pk}),m}^\chi$ transformation from [DHK⁺21]. This construction requires the following components: the message space $\mathcal{M} = \{0,1\}^\ell$ of DTRU.PKE, a hash function $H : \{0,1\}^* \rightarrow \{0,1\}^\kappa \times \{0,1\}^{\text{coins}}$ (where coins denotes the randomness length for DTRU.PKE.Enc), and a fixed-output-length function $ID : \mathcal{PK} \rightarrow \{0,1\}^\gamma$ with $\gamma \geq 256$ for security, which maps public keys to fixed-length prefixes. The complete DTRU.KEM algorithms are detailed in Fig. 2.

1.4 Parameter Specifications

Table 1 presents the algorithm instance parameters corresponding to different security strength levels. The DTRU scheme provides multiple security levels: DTRU-648¹ and DTRU-Light corresponds to 128-bit security, DTRU-768² to 192-bit, DTRU-1024 to 256-bit, DTRU-1536 to 384-bit, and DTRU-2048 to 512-bit security. Additionally, DTRU-Prime offers an alternative instantiation at 256-bit security level based on LPPNF.

In the table, parameters n , q , and q_2 represent the polynomial dimension, modulus, and compressed modulus, respectively. The distribution parameters (k_g, k_f) and (k_r, k_e) correspond to key generation (f, g) and encryption (r, e) , respectively. The (C, Q) values for LWE and NTRU columns indicate security levels against classical and quantum attacks, while δ denotes the decryption failure rate under the specified parameters.

Table 1: Parameters and Security Levels of DTRU Schemes

Scheme	n	q	q_2	(k_g, k_f)	(k_r, k_e)	LWE (C,Q)	NTRU (C,Q)	δ
DTRU	648	3457	2^9	(B_9, B_2)	(B_2, B_2)	(164, 144)	(164, 144)	$2^{-146.79}$
	768	3457	2^{10}	(B_4, B_3)	(B_4, B_2)	(198, 174)	(195, 171)	$2^{-185.56}$
	1024	3457	2^{10}	(B_5, B_2)	(B_3, B_2)	(271, 238)	(270, 237)	$2^{-190.48}$
	1536	3457	2^{10}	(B_3, B_2)	(B_2, B_1)	(411, 361)	(416, 365)	$2^{-195.50}$
	2048	3457	2^{10}	(B_5, B_1)	(B_2, B_1)	(567, 498)	(571, 501)	$2^{-204.10}$
DTRU-Light	512	769	2^8	(B_2, B_1)	(B_1, B_2)	(136, 120)	(132, 116)	$2^{-143.92}$
DTRU-Prime	1087	2017	2^{10}	(B_2, B_1)	(B_2, B_1)	(288, 253)	(280, 246)	$2^{-180.72}$

Table 2 presents the bandwidth of DTRU schemes, showing the ciphertext and public key sizes across different security levels. While DTRU prioritizes computational efficiency by not employing additional lossless public key compression techniques, we also provide a version with lossless compression for comparison. The numbers in parentheses in the ‘PK’ column represent the public key size after lossless compression. The compact design of DTRU is demonstrated by its efficient bandwidth utilization while providing strong security

¹DTRU-648 targets 128-bit quantum security.

²DTRU-768 targets 192-bit classical security, matching Kyber-768 (NIST’s primary recommended parameter level), and provides an additional recommended security level beyond those solicited by the cryptographic standard.

guarantees. DTRU-Light, considering its lightweight implementation, does not offer an additional lossless public key compression version and only provides parameters for reference. DTRU-Prime, since q is close to a power of two, does not require additional public key compression.

Table 2: Bandwidth of DTRU Schemes

Scheme	CT (bytes)	PK (bytes)	Error rate	Classical Security	Security Level
DTRU-648	729	972(954)	2^{-146}	164	128 bits
DTRU-Light	512	640(615)	2^{-143}	132	
DTRU-768	960	1152(1130)	2^{-185}	195	192 bits
DTRU-1024	1280	1536(1506)	2^{-190}	270	256 bits
DTRU-Prime	1359	1495	2^{-180}	280	
DTRU-1536	1920	2304(2259)	2^{-195}	411	384 bits
DTRU-2048	2560	3072(3012)	2^{-204}	567	512 bits

2 Design Principles

This section outlines the design philosophy and strategic considerations behind the DTRU scheme, covering the underlying mathematical foundations, parameter selection rationale, failure probability analysis, and additional design considerations.

2.1 Mathematical Foundation

DTRU is built upon the computational hardness of two well-established lattice problems: the Ring Learning With Errors (RLWE) problem and the NTRU problem. The security of DTRU relies on the assumption that these problems remain intractable even for quantum computers. The scheme achieves provable IND-CCA security through the Fujisaki-Okamoto transformation applied to the underlying NTRU-based public key encryption scheme.

Our primary instantiation employs tricyclotomic rings $(\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1))$, which provide NTT-friendly structures for efficient polynomial multiplication while maintaining strong security properties. Additionally, we offer DTRU-Prime instantiation on LPPNF (large-Galois-group prime-degree prime-ideal number field $\mathbb{Z}_q[x]/(x^n - x - 1)$) as a conservative defense against potential algebraic attacks, employing subring- and subfield-free structures, and we offer DTRU-Light instantiation on power-of-two cyclotomic rings $(\mathbb{Z}_q[x]/(x^n + 1))$ to provide a light-weight variant for resource-constrained devices.

2.2 Parameter Selection Strategy

The parameter selection strategy for DTRU employs a systematic methodology that carefully balances security, efficiency, and bandwidth considerations. This

process is structured around three interdependent optimization phases:

- **Security Parameter Optimization:** The polynomial dimension n and modulus q are jointly selected to achieve the target security level while enabling efficient NTT-based polynomial multiplication. The tricyclic rings and power-of-two cyclotomic rings are specifically chosen for its NTT-friendly properties, facilitating rapid arithmetic operations while providing strong security guarantees across multiple security levels (128-bit to 512-bit).
- **Bandwidth Efficiency Optimization:** The compressed modulus q_2 is optimized to strike an optimal balance between bandwidth efficiency and acceptable error rates during ciphertext compression. This parameter directly influences the trade-off between communication overhead and decryption reliability.
- **Noise Distribution Optimization:** Noise parameters are meticulously calibrated to achieve the optimal equilibrium between security strength and error tolerance. We deliberately avoid sparse noise distributions in favor of centered binomial distributions, employing distinct parameters (k_g, k_f) for key generation and (k_r, k_e) for encryption. These parameters are systematically adjusted to maintain robust security margins while ensuring practical decryption failure rates.

The parameter selection process follows a sequential optimization strategy: first, the ring parameters n and q are chosen to meet security requirements while maximizing NTT efficiency (by optimizing the factors of 2 and 3 in n , and ensuring $q - 1$ contains as many of these factors as possible); subsequently, noise parameters and the compression ratio q_2 are co-optimized to achieve the desired balance between bandwidth efficiency, error tolerance, and security strength.

This systematic methodology ensures that DTRU delivers robust security guarantees while maintaining competitive performance characteristics across all security levels.

2.3 Analysis of Error Rate

Theorem 1. *Let $\Psi_f, \Psi_g, \Psi_r, \Psi_e$ be distributions over the ring R_q , with f', g, r, e sampled accordingly. Let $p = 2$ (or $p = 1 - x^{n/2}$), $f = pf' + 1$, and $w = \text{Encode}(m) = (1 + x^{n/2})w'$ encode a message $m \in \mathcal{M}$, with ϵ denoting the rounding error from ciphertext compression and decompression. Define Err_i as the i -th octet of $gr + f(e + \epsilon) + f'w$ (or $gr + f(e + \epsilon) + f'w'$). If*

$$1 - \delta = \Pr \left[\|\text{Err}_i\|_{q,2}^2 + \|\text{Err}_{i+n/16}\|_{q,2}^2 < (q-1)^2/2, 1 \leq i \leq n/16 \right],$$

then the decryption failure rate is bounded above by δ .

Proof. Note that $w = \text{Encode}(m) = (1 + x^{n/2})w'$, where w' is a polynomial in R_q of degree less than $n/2$. The decryption algorithm yields $m' = \text{Decode}(c'f \bmod q)$.

Let $\epsilon = \left\lfloor \frac{q}{q_2} \left\lfloor \frac{q_2}{q} (hr + e + \frac{q+1}{2}w) \right\rfloor \right\rfloor - (hr + e + \frac{q+1}{2}w)$ be the rounding error. Then

$$c' = \left\lfloor \frac{q}{q_2} c \right\rfloor = \left\lfloor \frac{q}{q_2} \left\lfloor \frac{q_2}{q} (hr + e + \frac{q+1}{2}w) \right\rfloor \right\rfloor = hr + e + \frac{q+1}{2}w + \epsilon.$$

Since

$$\begin{aligned} \frac{q+1}{2}w \cdot f &= \frac{q+1}{2}w \cdot (pf' + 1) = \frac{q+1}{2}w \cdot pf' + \frac{q+1}{2}w \\ &= \begin{cases} f'w' + \frac{q+1}{2}w & p = 1 - x^{n/2} \text{ in } R_q = \mathbb{Z}_q[x]/(x^n + 1), \\ f'w + \frac{q+1}{2}w & p = 2 \text{ in other rings.} \end{cases} \end{aligned}$$

We have

$$c'f = \begin{cases} \underbrace{gr + f(e + \epsilon) + f'w' + \frac{q+1}{2}w}_{Err} & p = 1 - x^{n/2} \text{ in } R_q = \mathbb{Z}_q[x]/(x^n + 1), \\ \underbrace{gr + f(e + \epsilon) + f'w + \frac{q+1}{2}w}_{Err} & p = 2 \text{ in other rings.} \end{cases}$$

Let Err_i denote the i -th octet of Err , and u'_i the i -th octet of w' . The noise vector for the i -th scaled codeword $\frac{q+1}{2}(u'_i, u'_i)$ is $(Err_i, Err_{i+n/16})$. By the minimum distance property of scaled codewords, if

$$\|(Err_i, Err_{i+n/16})\|_{q,2}^2 = \|Err_i\|_{q,2}^2 + \|Err_{i+n/16}\|_{q,2}^2 < \left(\frac{q-1}{\sqrt{2}}\right)^2,$$

then the i -th codeword decodes correctly.

Hence, decryption succeeds when $\|Err_i\|_{q,2}^2 + \|Err_{i+n/16}\|_{q,2}^2 < \frac{(q-1)^2}{2}$ for all $1 \leq i \leq n/16$, and the failure probability is bounded above by δ . $\square$

To estimate the decryption failure probability δ , we adopt the independence assumption among coefficients, justified by the following two complementary arguments.

Theoretical justification. Lemma 3 in [LS25] shows that for large n , the covariance matrix of noise terms (e.g., gr and $f'e$) converges to a scaled identity matrix, precisely the form expected for independent normal distributions.

Precedent in established schemes. Kyber [SAB⁺20] and Scloud+ [WZZ⁺24] adopt analogous independence assumptions when estimating their failure rates, treating rounding errors as i.i.d. and independent of other terms (see also section 2.3 in [LS25]).

We stress that fully eliminating the independence assumption for high-dimensional lattice codes remains an open problem [PJH⁺25], and contemporary schemes such as NEV [ZFY23] and DAWN [LZL⁺25] rely on the same assumption. Under these circumstances, adopting the independence assumption is both a pragmatic necessity and a well-motivated choice.

Under this assumption, the failure probability calculation reduces to computing the tail probability of a chi-squared distribution. Applying the union bound:

$$\delta \leq \sum_{i=1}^{n/16} \Pr \left[\|Err_i\|_{q,2}^2 + \|Err_{i+n/16}\|_{q,2}^2 \geq \frac{(q-1)^2}{2} \right].$$

The squared norm follows a chi-squared distribution with 16 degrees of freedom:

$$\|Err_i\|_{q,2}^2 + \|Err_{i+n/16}\|_{q,2}^2 \sim \sigma_{Err}^2 \cdot \chi^2(16),$$

yielding the bound:

$$\delta \leq \frac{n}{16} \cdot \Pr \left[\chi^2(16) \geq \frac{(q-1)^2}{2\sigma_{Err}^2} \right].$$

The variance σ_{Err}^2 represents the variance of the coefficient of error distribution. This analysis ensures that DTRU maintains negligible failure rates (typically below 2^{-130}) across all security levels while providing strong security guarantees. Results of the failure rate are given in Table 1.

2.4 Additional Design Considerations

Several additional design principles are considered:

- **Resilience to DFR Manipulation:** While the double E_8 encoding structure $w = (\text{Encode}(m), \text{Encode}(m))$ introduces algebraic structure that could potentially be exploited by adversaries to craft messages inducing high decryption failure rates (DFR), we address this concern through careful design. For CCA-secure DTRU-KEM, the message m is actually random output by RO, which means adversary is hard to manipulate w . Moreover, we estimate that the parameter selection ensures even in worst-case scenarios the DFRs remain remarkably low (only by a $\sim 2^{20}$ factor than average-case).

These design principles collectively ensure that DTRU achieves its core objectives of versatility, compactness, simplicity, and robustness while providing provable security guarantees under standard cryptographic assumptions.

3 Security Analysis

3.1 Provable Security

3.1.1 IND-CPA Security of DTRU.PKE

Theorem 2. *Let $\Psi_f, \Psi_g, \Psi_r, \Psi_e$ be the distributions over the ring R_q and p be a invertible polynomial in R_q . Then, under the Decisional NTRU assumption and RLWE assumption, our DTRU.PKE scheme is IND-CPA secure in the standard*

model. That is, for any probabilistic polynomial-time adversary $\mathcal{A}$, there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ such that

$$\text{Adv}_{\text{DTRU.PKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{\Psi_f, \Psi_g}^{\text{DNTRU}}(\mathcal{B}) + \text{Adv}_{\Psi_r, \Psi_e}^{\text{RLWE}}(\mathcal{C}).$$

The IND-CPA security of DTRU.PKE reduces directly to the NTRU and RLWE assumptions via standard techniques.

Proof. Let $\mathcal{A}$ be an adversary against the IND-CPA security of our DTRU.PKE scheme. We prove the theorem by a sequence of games G_0, G_1, G_2 , where G_0 is the original IND-CPA game and G_2 is the random guess game. The security of the scheme is established by showing that G_0 and G_2 are indistinguishable from adversary $\mathcal{A}$'s view. Let S_i be the event that $\mathcal{A}$ wins in game G_i for $i = 0, 1, 2$. **Game G_0 .** This is the original IND-CPA game.

- **Key Generation.** The challenger generates $(h = g/f, f)$ by running the KeyGen algorithm, and sends h to $\mathcal{A}$.
- **Query.** The adversary $\mathcal{A}$ is allowed to make adaptive encryption queries to the challenger, who responds by running the Enc algorithm.
- **Challenge.** The adversary $\mathcal{A}$ submits two messages $m_0, m_1 \in \mathcal{M}$ to the challenger. The challenger chooses a random bit $b \in \{0, 1\}$, computes the ciphertext $c^* = \text{Enc}(pk, m_b)$ and sends it to $\mathcal{A}$. The adversary $\mathcal{A}$ outputs a guess b' of b and wins if $b' = b$.

We have $\text{Adv}_{\text{DTRU.PKE}}^{\text{IND-CPA}}(\mathcal{A}) = |\Pr[S_0] - \frac{1}{2}|$.

Game G_1 . This game is the same as G_0 except that the challenger generates h uniformly at random from R_q , while in G_0 , h is an NTRU instance $h = g/f$.

Therefore, if there exists a distinguisher $\mathcal{A}$ that can distinguish G_0 and G_1 , then we can construct an adversary $\mathcal{B}$ that breaks the Decisional NTRU assumption. Thus, we have

$$|\Pr[S_0] - \Pr[S_1]| \leq \text{Adv}_{\Psi_f, \Psi_g}^{\text{DNTRU}}(\mathcal{B}).$$

Game G_2 . This game is the same as G_1 except that in the encryption algorithm, the challenger replaces the term $hr + e$ with a uniformly random element u from R_q , i.e., the ciphertext is computed as $c = \left\lfloor \frac{q_2}{q} (u + \frac{q+1}{2} \text{Encode}(m)) \right\rfloor \bmod q_2$.

Since in G_1 , $hr + e$ is an RLWE instance, while in G_2 , u is uniformly random, if there exists a distinguisher $\mathcal{A}$ that can distinguish G_1 and G_2 , then we can construct an adversary $\mathcal{C}$ that breaks the RLWE assumption. Thus, we have

$$|\Pr[S_1] - \Pr[S_2]| \leq \text{Adv}_{\Psi_r, \Psi_e}^{\text{RLWE}}(\mathcal{C}).$$

Note that in G_2 u is uniformly random, the term $u + \frac{q+1}{2} \text{Encode}(m)$ is also random, so the ciphertext c^* is independent of the message. Therefore, we have $\Pr[S_2] = 1/2$.

Combining the above results, we have

$$\begin{aligned}
\text{Adv}_{\text{DTRU.PKE}}^{\text{IND-CPA}}(\mathcal{A}) &= \left| \Pr[S_0] - \frac{1}{2} \right| \\
&\leq |\Pr[S_0] - \Pr[S_1]| + |\Pr[S_1] - \Pr[S_2]| \\
&\leq \text{Adv}_{\Psi_f, \Psi_g}^{\text{DNTRU}}(\mathcal{B}) + \text{Adv}_{\Psi_r, \Psi_e}^{\text{RLWE}}(\mathcal{C}).
\end{aligned}$$

This completes the proof of the theorem. $\square$

3.1.2 IND-CCA Security of DTRU.KEM

Since DTRU.PKE is IND-CPA secure, by the security proof of $\text{FO}_{ID(pk),m}^\chi$ transformation in [DHK⁺21], we have the following theorem.

Theorem 3. *Let γ be the output length of ID and ℓ be the min entropy of $ID(pk)$, i.e. $\ell = H_\infty(ID(pk))$, where $(pk, sk) \leftarrow \text{DTRU.PKE.KeyGen}$. For any (quantum) adversary $\mathcal{A}$ that makes at most q_D decapsulation queries and q_H (quantum) random oracle queries against the IND-CCA security of DTRU.KEM, there exists a (quantum) adversary $\mathcal{B}$ with roughly the same running time as $\mathcal{A}$ such that:*

- In the random oracle model (ROM), $\text{Adv}_{\text{DTRU.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq$

$$2 \left(\text{Adv}_{\text{DTRU.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{q_H + 1}{|\mathcal{M}|} \right) + \frac{q_H}{2^\kappa} + (q_H + q_D)\delta + \frac{1}{2^\ell}$$

- In the quantum random oracle model (QROM), $\text{Adv}_{\text{DTRU.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq$

$$2\sqrt{q \text{Adv}_{\text{DTRU.PKE}}^{\text{IND-CPA}}(\mathcal{B})} + \frac{4q}{\sqrt{|\mathcal{M}|}} + \frac{1}{2^\ell} + \frac{4(q_H + 1)}{\sqrt{2^\kappa}} + 16q^2\delta + \frac{1}{|\mathcal{M}|},$$

where $q = q_H + q_D + 1$ and δ denotes the decryption failure probability of DTRU.PKE.

3.2 Concrete Security

This section analyzes the security of the DTRU scheme, which relies on the computational hardness of RLWE and NTRU problems. We evaluate their resilience against state-of-the-art cryptanalytic techniques.

3.2.1 Concrete Attacks on RLWE

To evaluate the security strength of the RLWE problem, we adopt the Core-SVP framework and analyze various attack methodologies. Since lattice reduction algorithms like BKZ cannot directly exploit the algebraic structure of RLWE, we first transform the RLWE instance into an equivalent LWE instance for analysis.

RLWE-to-LWE Transformation. Given an RLWE instance $(a, t = as + e) \in R_q \times R_q$, we transform it to an LWE instance as follows:

Let $\mathbf{a} \in \mathbb{Z}_q^n$ be the coefficient vector of polynomial a , and define the matrix $A \in \mathbb{Z}_q^{n \times n}$ as the matrix representation of multiplication by a in the ring R_q . Similarly, let $\mathbf{s}, \mathbf{e}, \mathbf{t} \in \mathbb{Z}_q^n$ be the coefficient vectors of s, e, t respectively. Then the RLWE instance becomes the LWE instance:

$$(A, \mathbf{t} = A\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{n \times n} \times \mathbb{Z}_q^n.$$

Core-SVP Methodology. BKZ-class attacks [CN11] represent the most effective methods against RLWE. These attacks transform the LWE problem into a short vector problem (SVP) on a lattice using embedding techniques (Kannan embedding [Kan87] and Bai-Galbraith embedding [BG14]), then solve the SVP using the BKZ lattice basis reduction algorithm.

The BKZ algorithm iteratively examines b -dimensional projected sublattices, solving the SVP on these sublattices using enumeration or sieving. The reduced basis vectors follow the geometric series assumption, enabling estimation of the block size b required to solve the SVP, which provides the security estimate.

Primal Attack. The primal attack reduces LWE to solving the unique shortest vector problem (u-SVP). Given $(A, \mathbf{t} = A\mathbf{s} + \mathbf{e})$, we construct an $(n + m + 1)$ -dimensional lattice:

$$\mathcal{L} = \{ \mathbf{x} \in \mathbb{Z}^{n+m+1} \mid (A_{[1:m]} \mid I_m \mid -\mathbf{t}_{[1:m]})\mathbf{x} \equiv \mathbf{0} \pmod{q} \}$$

where $m \leq n$. The lattice basis matrix is:

$$B = \begin{pmatrix} I_n & 0 & 0 \\ -A_{[1:m]} & qI_m & \mathbf{t}_{[1:m]} \\ 0 & 0 & 1 \end{pmatrix}$$

The unique shortest vector $\mathbf{v} = (\mathbf{s}^T \mid \mathbf{e}^T \mid 1)^T$ in $\mathcal{L}$ solves the LWE problem, with $\|\mathbf{v}\| \approx \sqrt{\alpha_1^2 n + \alpha_2^2 m}$.

Dual Attack. The dual attack finds short vectors in the dual lattice. Given $(A, \mathbf{t})$, we construct the dual lattice:

$$\mathcal{L}^* = \{ (\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^n \mid A_{[1:m]}^T \mathbf{x} = \mathbf{y} \pmod{q} \}$$

A short vector $(\mathbf{x}, \mathbf{y}) \in \mathcal{L}^*$ allows recovery of $\mathbf{s}_{[m+1:n]}$ through:

$$\langle \mathbf{x}, \mathbf{t} \rangle = \langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{[1:m]} \rangle + \langle A_{[m+1:n]}^T \mathbf{x}, \mathbf{s}_{[m+1:n]} \rangle.$$

In addition to BKZ-class attacks, we analyze hybrid attacks (effective for binary/ternary secrets), BKW attacks (requiring exponential samples), and algebraic attacks (Arora-Ge method).

Hybrid attack. Hybrid attack is an attack technique that combines lattice basis reduction and Meet In The Middle (MITM) attacks, which can efficiently target cases where the secret vector has binary or ternary distributions (i.e.,

selected from 0,1 or -1,0,1). Existing works have provided many heuristic and theoretical analyses of hybrid attacks [Wun19].

BKW attack. The composite methods proposed in [GJS15, KF15] extend the BKW method, making the BKW attack the most effective method for specific parameters. However, these methods require an exponential or sub-exponential number of samples.

Algebraic attack. Common algebraic attacks generally refer to Arora-Ge attack [AG11], but the number of samples required by the Arora-Ge attack depends on the error distribution, typically requiring an exponential or sub-exponential number of samples.

3.2.2 Concrete Attacks on NTRU

Similar to the RLWE instance, the NTRU instance $(h = f^{-1}g \bmod q) \in R_q$ can be viewed as a RLWE instance $(h, 0 = hf - g) \in R_q \times R_q$ with a zero target vector, which can be transformed into the corresponding LWE instances $(H', 0 = H'f - g) \in \mathbb{Z}_q^{n \times n} \times \mathbb{Z}_q^n$ for analysis.

While most attack methods on RLWE can be applied to NTRU, the dual attack [ACD⁺18] is not applicable. The dual attack relies on a non-zero target vector t' to recover the secret through the computation $\langle x, t' \rangle = \langle x, e \rangle + \langle y, s \rangle$. In the NTRU-to-LWE transformation, the target vector becomes zero, rendering this mechanism ineffective.

3.2.3 Concrete Security of DTRU

We employ the aforementioned attack methodologies to evaluate the security strength of the DTRU scheme using the Python script³ from [SAB⁺20]. Note that our estimate of the security strength is based on the model with classical progressive coefficient 0.292 and quantum progressive coefficient 0.2563. Results of the security strength are given in Table 1.

4 Benchmark

4.1 Performance Analysis

DTRU demonstrates significant performance advantages through several key design innovations:

Algebraic Structure Optimization: DTRU employs tricyclotomic rings $\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1)$ and power of two rings $\mathbb{Z}_q[x]/(x^n + 1)$, which support efficient Number Theoretic Transform (NTT) operations. Compared to traditional NTRU schemes, cyclotomic rings offer superior algebraic properties, enabling more efficient NTT decomposition and fewer modular reduction layers.

Incomplete NTT Technique: DTRU utilizes incomplete NTT with Cooley-Tukey algorithm for forward NTT and Gentleman-Sande algorithm for

³<https://github.com/pq-crystals/security-estimates>

inverse NTT. This approach maintains computational precision while significantly reducing the frequency of modular reduction operations. Different NTT transformation strategies are employed for various polynomial rings to ensure optimal performance across security levels.

Efficient Modular Reduction: The implementation employs Barrett reduction and Montgomery reduction instead of traditional modular operations. These algorithms utilize shift and multiplication operations to achieve modular reduction, avoiding expensive division operations and significantly improving computational efficiency.

Double E_8 Encoding Scheme: DTRU’s innovative double E_8 encoding combines NEV’s repetition code with extended Hamming code, achieving optimal error correction capability with a minimum Hamming distance of 8. The encoding maps 4 bits to 16 coordinates, providing robust error recovery while maintaining moderate decoding complexity, ensuring overall performance advantages.

Constant-Time Implementation: All critical operations are implemented in constant time to eliminate timing attack risks. Both Barrett and Montgomery reductions, along with the double E_8 decoding algorithm, are implemented using bitwise operations to ensure execution time independence from secret data.

4.2 Performance Testing

4.2.1 C Language Implementation (x86 Platform)

Implementation Details: The portable C implementation exclusively uses 16-bit and 32-bit integer arithmetic without floating-point operations. Polynomial coefficients are represented as signed 16-bit integers, and the scalable E_8 lattice decoder utilizes 32-bit integer arithmetic with constant-time implementation. All hash functions are implemented using SHA-3 family functions (SHAKE-256 and SHA3-512). Distribution sampling employs efficient bitwise operations for centered binomial.

Platform Configuration:

- **Processor:** Intel Core i5-12400F CPU
- **Clock Frequency:** 2.5GHz
- **x86 Instruction Set:** 64-bit base instruction set
- **Number of Cores:** 1
- **Memory:** 16GB
- **Operating System:** Ubuntu-20.04
- **Compiler:** GNU GCC 9.4.0
- **Compiler Flags:** gcc -std=c99 -Wpedantic -Wall -Wextra -O2

Performance Results: As shown in Table 3, the C language implementation demonstrates excellent performance across all security levels. The performance scales gracefully with increasing security levels, maintaining computational efficiency while ensuring robust cryptographic protection.

Table 3: Performance results (C and AVX2)

Scheme	Ref (CPU Cycles)			AVX2 (CPU Cycles)		
	KeyGen	Encap	Decap	KeyGen	Encap	Decap
DTRU-648	153809	80522	161932	44276	22043	48033
DTRU-768	142136	95394	173231	14538	15319	32806
DTRU-1024	181434	121786	234888	27080	29493	55577
DTRU-1536	234265	180105	367653	26708	26273	65131
DTRU-2048	426581	291726	516422	51105	59183	108685
DTRU-Light	66921	49800	94537	10795	12317	25926
DTRU-Prime	57574353	144513	292133	644910	32340	78920

4.2.2 AVX2 Vectorized Implementation

Implementation Details: The AVX2 implementation employs a unified vectorization strategy across all algebraic rings. Using 256-bit `ymm` registers, the implementation packs either 16 16-bit or 8 32-bit coefficients. Structured shuffle operations (`Shuffle8`, `Shuffle4`, `Shuffle2`, `Shuffle1`) progressively refine coefficient layouts to maintain in-register locality as butterfly strides grow. This approach ensures fully vectorized computation with minimal memory traffic.

Platform Configuration: Same x86 platform as the C implementation, utilizing AVX2 instruction set extensions for SIMD parallelism. Compiler flags: “`gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`”.

Performance Results: Table 3 highlights the significant performance improvements achieved through AVX2 vectorization. The AVX2 implementation demonstrates substantial speedup across all operations. These optimizations leverage SIMD parallelism to maximize computational throughput while maintaining algorithmic correctness.

4.2.3 ARM Cortex-M4 Benchmarking

We also develop an optimized ARM Cortex-M4 implementation of DTRU and benchmark it on the STM32F4DISCOVERY board, which is equipped with an ARM Cortex-M4F core running at 24 MHz. The firmware is cross-compiled with `arm-none-eabi-gcc` from the GNU ARM Embedded Toolchain and the corresponding `binutils`. The evaluation includes three variants. The `ref` variant is the reference implementation and serves as the baseline for correctness and

comparison. The `m4fspeed` variant is optimized for speed on ARM Cortex-M4, while the `m4fstack` variant is optimized for stack usage. For each parameter set, namely DTRU-Light, DTRU-648, DTRU-768, DTRU-1024, DTRU-1536, DTRU-2048, and DTRU-Prime, we run the tests on the `stm32f4discovery` platform with 1,000 executions for each measurement point. The speed benchmark measures the complete DTRU KEM flow, including `kem_keygen`, `kem_enc`, and `kem_dec`. For benchmarking, the speed test and the stack-usage test each use 1,000 repetitions for every parameter set and KEM operation, and all clock cycle counts and stack values reported below are averages over these 1,000 measurements. The evaluation version is the one without compressed public keys.

The speed results are shown in Table 4. Compared with the reference implementation, the ARM Cortex-M4 speed-optimized implementation reduces the total measured cycle count by $1.83\times$, $1.49\times$, $1.85\times$, $1.68\times$, $1.94\times$, $1.33\times$, and $1.01\times$ for DTRU-Light, DTRU-648, DTRU-768, DTRU-1024, DTRU-1536, DTRU-2048, and DTRU-Prime, respectively, where the total is computed as the sum of key generation, encapsulation, and decapsulation. The stack-usage results are shown in Table 5. The ARM Cortex-M4 stack-optimized implementation reduces the maximum observed stack usage by 44.6%, 43.0%, 41.0%, 41.3%, 43.3%, 42.5%, and 15.6% for the same parameter sets, respectively, compared with the corresponding reference implementation.

Table 4: Speed of DTRU on ARM Cortex-M4 (Unit: clock cycle).

Parameter Set	Scheme	KeyGen	Encaps	Decaps
DTRU-Light	ref	464,845	447,286	887,039
	m4fspeed	273,796	238,641	469,766
DTRU-648	ref	1,174,756	676,426	1,333,159
	m4fspeed	911,459	413,127	806,561
DTRU-768	ref	990,982	867,798	1,642,410
	m4fspeed	551,022	477,117	861,066
DTRU-1024	ref	1,354,139	1,038,823	2,040,310
	m4fspeed	854,615	612,515	1,175,176
DTRU-1536	ref	1,706,021	1,659,292	3,185,080
	m4fspeed	918,191	870,543	1,595,825
DTRU-2048	ref	3,231,574	2,386,203	4,545,928
	m4fspeed	2,181,919	1,898,264	3,584,478
DTRU-Prime	ref	219,958,105	1,186,221	2,433,139
	m4fspeed	219,541,044	769,126	1,593,260

Table 5: Stack usage of DTRU on ARM Cortex-M4 (Unit: bytes).

Parameter Set	Scheme	KeyGen	Encaps	Decaps
DTRU-Light	ref	5,632	6,776	7,492
	m4fstack	3,260	3,596	4,148
DTRU-648	ref	8,608	9,004	9,752
	m4fstack	4,672	4,892	5,556
DTRU-768	ref	9,136	10,664	11,744
	m4fstack	4,732	5,928	6,928
DTRU-1024	ref	12,240	13,912	15,304
	m4fstack	6,364	7,660	8,980
DTRU-1536	ref	17,416	20,088	22,168
	m4fstack	9,328	10,568	12,568
DTRU-2048	ref	23,956	26,712	29,424
	m4fstack	12,728	14,172	16,924
DTRU-Prime	ref	39,500	41,828	43,300
	m4fstack	34,276	35,260	36,548

4.2.4 Transmission and Storage Overhead

DTRU achieves efficient bandwidth utilization with compact key and ciphertext sizes. The storage requirements are summarized in Table 6. While DTRU prioritizes computational efficiency by not employing additional lossless public key compression techniques, we also provide a version with lossless compression for comparison. The numbers in parentheses in the ‘PK’ column represent the public key size after lossless compression. DTRU-Light, considering its lightweight implementation, does not offer an additional lossless public key compression version and only provides parameters for reference. DTRU-Prime, since q is close to a power of two, does not require additional public key compression.

Table 6: Bandwidth of DTRU schemes

Scheme	Public Key (bytes)	Ciphertext (bytes)	Secret Key (bytes)
DTRU-648	972(954)	729	1328(1310)
DTRU-768	1152(1130)	960	1568(1546)
DTRU-1024	1536(1506)	1280	2080(2050)
DTRU-1536	2304(2259)	1920	3136(3091)
DTRU-2048	3072(3012)	2560	3904(3844)
DTRU-Light	640(615)	512	864(839)
DTRU-Prime	1495	1359	1935

5 Feature Declaration

DTRU exhibits several distinctive features that position it as a superior post-quantum cryptographic solution:

- **Algorithmic Innovation:** DTRU introduces a novel double E_8 encoding scheme that combines NEV’s repetition code with an extended Hamming-style code derived from the E_8 lattice. The encoding maps 4 bits to 16 coordinates and achieves a minimum Hamming distance of 8, providing robust error resilience with modest decoding complexity while maintaining computational efficiency. This work has been accepted as paper #773 at ESORICS 2026 ⁴, titled “DTRU: A Versatile, Compact, Simple, and Robust NTRU KEM with Double E_8 Encoding”, and a companion paper on DTRU-Light’s lightweight deployment appears at CHES 2026 [DKZ⁺26].
- **Algorithmic Simplicity:** The design philosophy emphasizes simplicity through standardized cryptographic primitives and straightforward implementation. Key generation avoids computing the inverse of f modulo p and only requires its inverse modulo q , a notable simplification over conventional NTRU variants. DTRU employs conventional compression functions ($\mathbb{Z}_q \rightarrow \mathbb{Z}_{q_2}$) rather than complex coefficient packing schemes, and DTRU-Light omits public-key compression entirely, further reducing implementation complexity. The decoding algorithm follows maximum likelihood principles with clear mathematical foundations. These design choices yield a compact codebase that facilitates auditability and deployment on resource-constrained platforms.
- **Conservative Security Design:** DTRU adopts a security-first philosophy grounded in conservative parameter choices. The scheme avoids sparse noise distributions, instead utilizing dense, centered binomial sampling to prevent key recovery attacks that exploit sparsity. Key generation follows a standard procedure without additional filtering, ensuring no unnecessary reduction of the key space. DTRU-648 is designed to satisfy both the classical and post-quantum 128-bit security targets (NIST Category 1), ensuring robust protection against both classical and quantum adversaries. Furthermore, DTRU’s parameter selection considers LPPNF rings to mitigate structural attacks related to subfield vulnerabilities.
- **Lightweight Implementation & Deployment Friendliness:** DTRU-Light is specifically tailored for ultra-constrained devices. By combining small NTT-friendly moduli, low-degree polynomials, and the compact double E_8 encoding, DTRU-Light achieves minimal memory footprint and low stack usage, making it deployable on extremely resource-limited platforms such as 8-bit AVR sensor nodes where most NIST-standardized KEMs cannot fit. As confirmed in the aforementioned CHES 2026 companion

⁴<https://sites.google.com/di.uniroma1.it/esorics2026/program/accepted-papers>

paper [DKZ⁺26], this lightweight design enables both pure post-quantum and hybrid key exchange on heavily constrained microcontrollers.

- **Algorithmic Flexibility:** DTRU demonstrates exceptional flexibility through multiple ring instantiations. The primary implementation uses tricyclotomic rings $(\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1))$ supporting various security levels, while DTRU-Prime provides a conservative alternative on LPPNF rings $(\mathbb{Z}_q[x]/(x^n - x - 1))$ and DTRU-Light offers a lightweight variant for resource-constrained devices on power-of-two cyclotomic rings $(\mathbb{Z}_q[x]/(x^n + 1))$. Notably, DTRU-648 is designed to achieve both classical and quantum 128-bit security (NIST Category 1), while DTRU-768 targets higher security levels and aligns with the international mainstream Kyber-768 for seamless interoperability. This flexibility enables the scheme to meet rigorous post-quantum security requirements across multiple security tiers while supporting diverse ring structures.
- **Algorithmic Compatibility:** The scheme maintains full compatibility with existing cryptographic infrastructure through standard NTRU-based construction. In particular, DTRU-768 is designed to align with Kyber-768, sharing comparable parameter sizes and security levels, which facilitates seamless integration and interoperability within established protocol ecosystems.
- **Algorithmic Extensibility:** The modular architecture supports seamless extension to higher security levels. DTRU parameters scale gracefully from 128-bit to 512-bit security, with consistent performance characteristics across the security spectrum. The double E_8 encoding framework can be adapted to future cryptographic requirements without fundamental architectural changes.
- **Cross-Platform Performance Advantages:** Comprehensive benchmarking demonstrates DTRU’s superior performance across diverse computing platforms. On x86-64 systems, DTRU achieves $8.76\text{--}15.69\times$ speedup over traditional NTRU schemes and $1.05\text{--}1.32\times$ improvement over Kyber in reference implementations. With AVX2 optimizations, performance gains remain substantial at $3.84\text{--}6.28\times$ over NTRU and $1.10\text{--}1.22\times$ over Kyber. On ARM Cortex-M4 embedded systems, DTRU consistently outperforms Kyber with $1.06\text{--}1.62\times$ speedup, validating its efficiency across resource-constrained environments.

References

- [ACD⁺18] Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the {LWE, NTRU} schemes! In *SCN*, volume 11035 of *Lecture Notes in Computer Science*, pages 351–367. Springer, 2018.

- [AG11] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In *ICALP (1)*, volume 6755 of *Lecture Notes in Computer Science*, pages 403–415. Springer, 2011.
- [BG14] Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary LWE. In *ACISP*, volume 8544 of *Lecture Notes in Computer Science*, pages 322–337. Springer, 2014.
- [CN11] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In *ASIACRYPT*, volume 7073 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2011.
- [DHK⁺21] Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, and Gregor Seiler. Faster lattice-based kems via a generic fujisaki-okamoto transform using prefix hashing. In *ACM CCS 2021*, pages 2722–2737, 2021.
- [DKZ⁺26] Yifan Dong, YoungBeom Kim, Jieyu Zheng, Zhichuang Liang, Boyue Fang, Seog Chung Seo, Máire O’Neill, and Yunlei Zhao. Lightweight PQ KEM and hybrid MQTT protocol for 8-bit AVR sensor nodes. *IACR Cryptol. ePrint Arch.*, 2026:248, 2026.
- [GJS15] Qian Guo, Thomas Johansson, and Paul Stankovski. Coded-bkw: Solving LWE using lattice codes. In *CRYPTO (1)*, volume 9215 of *Lecture Notes in Computer Science*, pages 23–42. Springer, 2015.
- [Kan87] Ravi Kannan. Minkowski’s convex body theorem and integer programming. *Math. Oper. Res.*, 12(3):415–440, 1987.
- [KF15] Paul Kirchner and Pierre-Alain Fouque. An improved BKW algorithm for LWE with applications to cryptography and lattices. In *CRYPTO (1)*, volume 9215 of *Lecture Notes in Computer Science*, pages 43–62. Springer, 2015.
- [LS25] Shuiyin Liu and Amin Sakzad. Lattice codes for crystals-kyber. *Des. Codes Cryptogr.*, 93(8):3181–3205, 2025.
- [LZL⁺25] Yijian Liu, Yu Zhang, Xianhui Lu, Yao Cheng, and Yongjian Yin. DAWN: smaller and faster NTRU encryption via double encoding. In *ASIACRYPT (3)*, volume 16247 of *Lecture Notes in Computer Science*, pages 396–427. Springer, 2025.
- [PJH⁺25] Thales B. Paiva, Marcos A. Simplicio Jr., Syed Mahbub Hafiz, Bahattin Yildiz, Eduardo Lopes Cominetti, and Henrique S. Ogawa. Tailorable codes for lattice-based kems with applications to compact ML-KEM instantiations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(3):139–163, 2025.

- [SAB⁺20] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, and Damien Stehlé. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [Wun19] Thomas Wunderer. A detailed analysis of the hybrid lattice-reduction and meet-in-the-middle attack. *J. Math. Cryptol.*, 13(1):1–26, 2019.
- [WZZ⁺24] Anyu Wang, Zhongxiang Zheng, Chunhuan Zhao, Zhiyuan Qiu, Guang Zeng, Ye Yuan, Changchun Mu, and Xiaoyun Wang. Scloud⁺: An efficient lwe-based KEM without ring/module structure. In *SSR*, volume 15559 of *Lecture Notes in Computer Science*, pages 147–174. Springer, 2024.
- [ZFY23] Jiang Zhang, Dengguo Feng, and Di Yan. NEV: faster and smaller NTRU encryption using vector decoding. In *ASIACRYPT (7)*, volume 14444 of *Lecture Notes in Computer Science*, pages 157–189. Springer, 2023.

A Exploration of Sparse-Noise Parameters for the Power-of-Two Ring

In the main body of this paper, we adopt a conservative parameter selection strategy: most recommended parameters employ centered binomial distributions with parameter $k \geq 2$ (i.e., B_2 or above), deliberately avoiding sparse noise distributions. This conservative choice prioritizes security against key recovery attacks that exploit noise sparsity, ensuring robust security margins across all security levels.

In this appendix, we explore alternative parameter sets for DTRU-Light-1024 and DTRU-Light-2048 instantiated on the power-of-two cyclotomic rings $(\mathbb{Z}_q[x]/(x^n + 1))$, where we relax the noise distribution to allow very sparse noise (e.g., B_1 or ternary distributions with small Hamming weight). As shown in Tables 7⁵, under such sparse-noise regimes, the bandwidth of DTRU can be further optimized while maintaining correctness. Notably, even with these aggressive parameter choices, DTRU’s ciphertext and public key sizes remain competitive with those of NEV [ZFY23] and DAWN [LZL⁺25], while offering the full benefits of the double E_8 encoding framework.

We stress that these sparse-noise parameters are presented for exploratory and comparative purposes only; the parameters recommended in the main text

⁵ $\mathcal{T}_k$ denotes each coefficient takes value 1 with probability k/n , -1 with probability k/n , and 0 otherwise.

(Tables 2 and 6) remain our primary and conservative recommendation for deployment.

Table 7: Sparse-noise parameters for DTRU-Light

n	q	q_2	(k_g, k_f)	(k_r, k_e)	LWE (C,Q)	NTRU (C,Q)	δ	(CT,PK) (bytes)
1024	769	2^8	$(B_1, \mathcal{T}_{128})$	$(\mathcal{T}_{192}, B_1)$	(264,236)	(261,229)	$2^{-211.81}$	(1024,1280)
	769	2^8	$(B_2, \mathcal{T}_{96})$	$(\mathcal{T}_{192}, B_1)$	(264,236)	(265,233)	$2^{-201.97}$	(1024,1280)
2048	769	2^8	$(B_1, \mathcal{T}_{162})$	$(\mathcal{T}_{162}, \mathcal{T}_{128})$	(548,488)	(537,477)	$2^{-247.32}$	(2048,2560)
	769	2^8	$(B_1, \mathcal{T}_{192})$	$(\mathcal{T}_{162}, \mathcal{T}_{128})$	(548,488)	(551,484)	$2^{-211.60}$	(2048,2560)
	769	2^8	$(B_1, \mathcal{T}_{192})$	$(\mathcal{T}_{192}, \mathcal{T}_{128})$	(564,495)	(551,484)	$2^{-202.85}$	(2048,2560)